

```
[OpenMP compile/run]: gcc -fopenmp hic -o hi; export
OMP_NUM_THREADS=8; ./hi
Makefile: gcc -c -fopenmp hic; gcc -fopenmp hio -o hi
[OpenMP hello]: #include <omp.h>
#include <stdio.h>
int main()
{
#pragma omp parallel
{
printf("Hello from thread %d\n",
omp_get_thread_num());
}
}

[OpenMP cpu mapping]: #include <sched.h>
int sched_getcpu(void);
#pragma omp parallel
{
printf("OMP %03d/%03d -> hwthread %03d\n",
omp_get_thread_num(), omp_get_num_threads(),
sched_getcpu());
#pragma omp master
printf("only thread %03d prints this\n",
omp_get_thread_num());
}

[OpenMP set thread count in code]: unsigned int
Nthrds=0;
sscanf(argv[1],
"%d", &Nthrds);
omp_set_dynamic(
0);
omp_set_num_threads(Nthrds);
[OpenMP affinity]: export OMP_PLACES=threads; export
OMP_PROC_BIND=close; export
OMP_NUM_THREADS=128
/hi2 | sort -n -k 7
[OpenMP parallel for template]: #pragma omp parallel
for default(none) \
shared(n,x,y) private(i)
for (i=0; i<n; i++)
x[i] += y[i];
[OpenMP parallel region + for]: #pragma omp parallel
if(n>limit) default(none)
\
shared(n,a,b,c,x,y,z) private(f,i,scale)
{
f = 1.0; // every thread executes. #pragma
omp for nowait
for(i=0; i<n; i++) z[i] = x[i] + y[i]; #pragma omp for
nowait
for(i=0; i<n; i++) a[i] = b[i] + c[i]; #pragma omp barrier;
scale = sum(a,0,n) + sum(z,0,n) + f;
}

[OpenMP barrier]: for(i=0; i<N; i++) a[i]=b[i]+c[i];
#pragma omp barrier
for(i=0; i<N; i++) d[i]=a[i]+b[i];
Need barrier if 2nd loop reads a[] written by 1st.
[OpenMP nowait form]: #pragma omp parallel
default(none) shared(n,a,b,c,d)
private(i)
{
#pragma omp for nowait; for(i=0; i<n; i++) d[i] = 1.0/c[i];
#pragma omp for nowait
for(i=0; i<n-1; i++) b[i] = (a[i]+a[i+1])/2; } // implied barrier
at end of parallel region
[OpenMP sections]: #pragma omp parallel default(none)
shared(n,a,b,c,d) private(i)
{
#pragma omp sections nowait
{
#pragma omp section; for(i=0; i<n; i++) a[i] = 1.0/c[i];
#pragma omp section
for(i=0; i<n-1; i++) b[i] = (a[i]+a[i+1])/2; }
}

[OpenMP single]: #pragma omp parallel { #pragma omp
single nowait init_once();
work_all_threads(); } /* single has
implicit barrier unless nowait */
[OpenMP master]: #pragma omp parallel { #pragma
omp master printf("TO only\n"); } /*
master = thread 0, no implicit barrier
*/
[OpenMP task skeleton]: #pragma omp parallel {
#pragma omp single {
for(i=0; i<N; i++) #pragma omp
task firstprivate(i) work(i);
#pragma omp taskwait } }
[OpenMP if clause]: #pragma omp parallel if(n >
threshold) shared(n,x,y) private(i) /*
false -> serial execution of region */
[OpenMP schedule static]: #pragma omp parallel for
schedule(static) /* fixed
chunks; low overhead; good if
iterations equal cost */
[OpenMP schedule dynamic]: #pragma omp parallel for
schedule(dynamic,4) /*
threads grab chunks of 4;
good if uneven work;
overhead higher */
[OpenMP collapse]: #pragma omp parallel for collapse(2)
private(i,j) shared(A,B,C)
for(i=0; i<N; i++) for(j=0; j<M; j++)
C[i*M+j]=A[i]*B[j];
[OpenMP ordered]: #pragma omp parallel for ordered
for(i=0; i<N; i++){ work(i); #pragma
omp ordered printf("%d\n",i); }
[OpenMP data attrs]: shared(x): one object; all threads
see same address. private(x): one
uninitialized x per thread.
firstprivate(x): initialize private x
from original. lastprivate(x): value
from lexically last iteration copied
out. default(none): force
shared/private/reduction clauses.
[OpenMP private gotcha]: #pragma omp parallel
private(private)
{
sum = 0.0; // private variables are uninitialized!
}
private values do not survive outside the parallel region.
```

```
[OpenMP reduction syntax]: #pragma omp parallel for
reduction(+)
for(unsigned long long i=1; i<=N; ++i)
j += i; Correct check: N*(N+1)/2; Allowed common ops: +
- * & | ^ && || min max
[OpenMP reduction timing]: double t0 =
omp_get_wtime();
#pragma omp parallel for reduction(+,S)
for(int j=0; j<=n; ++j) S += A[j];
double dt = omp_get_wtime() - t0; printf("%2e s\n", dt);
[OpenMP local-atomic]: #pragma omp parallel { double
local=0; #pragma omp for
nowait for(i=0; i<N; i++)
local+=a[i]; #pragma omp
atomic total+=local; }
[OpenMP max reduction]: #pragma omp parallel for
reduction(maxmx)
for(i=0; i<N; i++) if(a[i]>mx)
mx=a[i];
[OpenMP race fix: atomic]: #pragma omp atomic
a[indx[i]] += b[i]; Atomic = only load/store/update atomic;
lighter than critical.
[OpenMP race fix: critical]: #pragma omp critical
{
sum += a[i];
}
Critical = all code by one thread at a time; costly.
[OpenMP shared bug]: Wrong: shared sum with sum +=
a[i] in parallel for. Fix:
reduction(+,sum), atomic for one
update, or critical if update is
complex.
[OpenMP DGEMM inner loops]: #pragma omp parallel
for private(tmp,i,j,k)
for (i=0; i<Ndim; i++){
for (j=0; j<Mdim; j++){
tmp = 0.0; for(k=0; k<Pdim; k++){
*(A+(i*Ndim+k)) * *(B+(k*Pdim+j));
*(C+(i*Ndim+j)) = tmp; }
}
}
Private tmp,i,j,k; initialize tmp inside loop.
[OpenMP nested loops rule]: Parallelize outer loop when
each iteration has enough
work; privatize inner-loop
index. #pragma omp
parallel for private(i,tmp)
shared(A,B,C).
[OpenMP speedup code]: double S_n = T1 / Tn;
if(S_n > 1) faster;
if(S_n < 1) slowdown; // overhead dominates small N
Run powers of two threads: 1,2,4,8,16,32,64,128.
[OpenMP timing]: t0=omp_get_wtime(); /* region */
t1=omp_get_wtime();
printf("%3e\n",t1-t0); Do not time
malloc/print unless asked.
[OpenMP false sharing fix]: typedef struct { double x;
char pad[64]; } Padded;
Padded
*sum=malloc(n*threads,sizeof(
Padded)); sum[idx] += local;
[OpenMP rand issue]: rand() has shared state and may
serialize or race; for deterministic
thread-local random values use
rand_r(&seed) with private seed.
[OpenMP make run loop]: for t in 1 2 4 8 16 32 64 128;
do export
OMP_NUM_THREADS=$t;
./prog; done
[OpenMP build flags]: Missing -fopenmp: pragmas are
ignored as comments and omp.*
symbols may fail at link. Compile
and link with -fopenmp.
[OpenMP Riemann setup]: const unsigned int n=(unsigned
int)1e9; const double a=0,
b=1; const double h=(b-a)/n;
double S=0;
double *A=(double*)calloc(n+1,sizeof(double));
[OpenMP Riemann work split]: #pragma omp parallel
{
int id = omp_get_thread_num(); unsigned long long
istart = id*N/Nthrds;
unsigned long long iend = (id+1)*N/Nthrds; if(id ==
Nthrds-1) iend = n;
for(unsigned long long i=istart; i<iend; ++i); A[i] = f(a +
i*h);
}
#pragma omp parallel for reduction(+,S)
for(int j=0; j<=n; ++j) S += A[j];
[OpenMP rectangle integration]: double area = 0.0,
dx=(b-a)/N;
#pragma omp parallel for reduction(+,area)
for(int i=0; i<N; i++){
double x = a + (i+0.5)*dx; area += f(x)*dx;
}
[OpenMP Simpson 1/3 skeleton]: // N even, h=(b-a)/N;
sum = f(a) + f(b);
#pragma omp parallel for reduction(+,sum)
for(int i=1; i<N; i++){
double x = a + i*h; sum += (%2 ? 4.0 : 2.0)*f(x);
}
area = h*sum/3.0;
[OpenMP chunk math]: id=omp_get_thread_num();
nt=omp_get_num_threads();
istart=id*N/nt; iend=(id+1)*N/nt;
if(id==nt-1) iend=N;
[Gaussian elimination row update]: // zero A[j][i] using
pivot row i
for(size_t i=1; i<=N; i++)
for(size_t j=i+1; j<=N; j++){
double factor = a[j][i]/a[i][i]; for(size_t k=i; k<=N-1; k++)
a[j][k] -= a[i][k]*factor; }
[Back substitution]: for(int i=N; i>=1; i--){
x[i] = a[i][N+1]; for(int j=i+1; j<=N; j++) x[i] -= a[i][j]*x[j];
x[i] /= a[i][i];
}
```

```
[Forward substitution]: for(i=1; i<=N; i++){ y[i]=b[i];
for(j=1; j<i; j++) y[i]-=L[i][j]*y[j];
y[i]/=L[i][i]; }
[LU solve order]: A=LU; solve Ly=b, then Ux=y. In
triangularization, pivot row i is known
before updating rows j>i.
[Common OpenMP answer checklist]: 1 include <omp.h>;
2 set
OMP_NUM_THRE
ADS or
omp_set_num_thr
eads(); 3 identify
independent
loop(s)
4 mark shared(.) and private(.); 5 use reduction for
accumulators
6 use barrier/critical/atomic/lock only when needed; 7
initialize private accumulators inside region
[OpenMP node struct with lock]: typedef struct node {
int value; struct node *next, *prev; omp_lock_t lock;
} Node;
Node *head = calloc(1,sizeof(Node)); head->value=0;
head->next=NULL; head->prev=NULL;
omp_init_lock(&head->lock);
[OpenMP sequential list insert]: for(k=1; k<=N; k++){
prev=head; p=head->next; value=rand()%(1000+1);
while(pl=NULL){
if(p->value <= value) break; prev=p; p=p->next; };
newNode=calloc(1,sizeof(Node));
newNode->value=value; newNode->next=p;
newNode->prev=prev; newNode->prev->next=newNode;
omp_init_lock(&newNode->lock); if(p)
p->prev=newNode;
}
[OpenMP parallel list insert idea]: #pragma omp parallel
for
private(k,value,
newNode,p,prev)
for(k=1; k<=N; k++){
prev=head; p=head->next; value=rand()%(1000+1);
// while searching: lock prev; lock p; unlock old prev; //
on break: maintain locks on prev and p
// update prev->next and p->prev while both are locked;
// unlock prev; if(p) unlock p;
}
[OpenMP lock functions]: omp_lock_t L;
omp_init_lock(&L);
omp_set_lock(&L); // waits,
caller owns lock
/* protected pointer update */; omp_unset_lock(&L); //
release;
omp_destroy_lock(&L);
Do NOT omp_set_lock twice from same owner -> deadlock
[OpenMP linked-list insert locked]: omp_set_lock(&
prev->lock);
if(p) omp_set_lock(&p->lock);
newNode = calloc(1,sizeof(Node)); newNode->value = value;
newNode->next = p; newNode->prev = prev;
prev->next = newNode; omp_init_lock(&newNode->lock);
if(p) p->prev = newNode;
omp_unset_lock(&prev->lock);
if(p) omp_unset_lock(&p->lock);
[OpenMP in-order traversal]: p = head;
while(p != NULL){
printf("%03d ", p->value); p = p->next;
}
printf("\n");
[OpenMP lock hand-over-hand]: Search list: lock current
node before reading
next; lock next before
releasing current;
always lock in forward
order to avoid
deadlock.
[OpenMP deadlock pattern]: T0 locks A then waits B, T1
locks B then waits A ->
deadlock Rule: global lock
order or one lock at a
time.
[OpenMP ptr insert]: new->next=p; new->prev=prev;
prev->next=new; if(p) p->prev=new;
Required locks: prev for prev->next
and p for p->prev.
[CUDA compile as .cu]: nvcc VecAddCUDA2.cu -o
VecAddCUDA2; nvcc -x cu
VecAddCUDA3.cu -o
VecAddCUDA3
nvcc --lineinfo main.cu -o main
cuda-memcheck ./program
ncu -f --kernel-id :kernel:1 -o out executable
[CUDA function qualifiers]: __global__ void K(.) // called
host, runs device, returns
void
__device__ float f() // called device, runs device; __host__
float h() // called host, runs host
__host__ __device__ float both0(.,.)
[CUDA variable scope]: int LocalVar; //
register; thread
__shared__ int SharedVar; // shared, block
__device__ int GlobalVar; // global, grid/app
__constant__ int ConstantVar; // constant, grid/app
__shared__ float ds[TILE_WIDTH][TILE_WIDTH];
[CUDA thread hierarchy]: grid -> blocks -> threads.
blockIdx identifies block;
threadIdx identifies thread
inside block; blockDim gives
block size.
[CUDA 1D index guard]: __global__ void kernel(int *a, int
N){
int i = blockIdx*x*blockDim.x + threadIdx.x; if(i < N) a[i]
= i;
}
[CUDA blocks formula]: int threads = 128; int blocks = (N
+ threads - 1) / threads;
kernel<<blocks, threads>>>(a,
N);
For N=4097, threads=128 -> blocks=33.
```

```
[CUDA H2D/D2H aliases]: #define H2D
cudaMemcpyHostToDevice;
#define D2H
cudaMemcpyDeviceToHost //
HIP equivalents:
#define H2D hipMemcpyHostToDevice; #define D2H
hipMemcpyDeviceToHost
[CUDA error macro]: #define cudaCheckError() { \;
cudaError_t e=cudaGetLastError();
\; if(e!=cudaSuccess){ \;
printf("Cuda failure %s%d: %s\n",__FILE__,__LINE__, \;
cudaGetErrorString(e)); exit(EXIT_FAILURE); \; } \;
}
[CUDA dense copy skeleton]: int bytes=N*sizeof(float);
cudaMalloc(&d,bytes);
cudaMemcpy(d,h,bytes,
H2D);
kernel<<((N+255)/256,
256>>>)(d,N);
cudaMemcpy(h,d,bytes,
D2H); cudaFree(d);
[CUDA vecAdd host/device workflow]: void vecAdd(float
*h,A,float
*h,C,int
n){
float *d_A,*d_B,*d_C;
cudaMalloc((void**) &d_A,n*sizeof(float));
cudaMalloc((void**) &d_B,n*sizeof(float));
cudaMalloc((void**) &d_C,n*sizeof(float));
cudaMemcpy(d_A,h,A,n*sizeof(float),
cudaMemcpyHostToDevice);
cudaMemcpy(d_B,h,B,n*sizeof(float),
cudaMemcpyHostToDevice);
kernel<<1,3>>>(&d_A,&d_B,&d_C,n);
cudaMemcpy(h,C,d_C,n*sizeof(float),
cudaMemcpyDeviceToHost);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
[CUDA vecAdd kernel]: __global__ void kernel(float
*d_A,float
*d_B,float
*d_C,int n){
int i = blockIdx*x*blockDim.x + threadIdx.x; if(i < n)
d_C[i] = d_A[i] + d_B[i];
}
[CUDA managed memory]: int *a;
cudaMallocManaged(&a, N*sizeof(int));
kernel<<blocks,threads>>>(a,N);
cudaDeviceSynchronize(); // CPU sees updated values
for(int i=0; i<10; i++) printf("%d\n",a[i]);
cudaFree(a);
[CUDA copy sizes]: cudaMemcpy(d,h,N*sizeof(float),
cudaMemcpyHostToDevice); use
sizeof(type)*number_of_elements, not
sizeof(pointer).
[CUDA saxpy kernel]: __global__ void saxpy(int n,float
a,float*x,float*y){ int
i=blockIdx*x*blockDim.x+
threadIdx.x; if(i<n) y[i]=x[i]*y[i]; }
[CUDA launch config]: dim3 block(16,16); dim3
gnd((w+block.x-1)/block.x,(h+
block.y-1)/block.y);
kernel<<gnd,block>>>(...);
[CUDA 2D grid launch]: dim3 DimBlock(16,16,1);
dim3 DimGrid((n-1)/16 + 1, (m-1)/16 + 1, 1);
imageKernel<<DimGrid,DimBlock>>>(d_Pin,d_Pout,m,n);
// Col maps x, Row maps y
[CUDA 2D row/col index]: int Col = blockIdx.x *
blockDim.x + threadIdx.x; int
Row = blockIdx.y * blockDim.y
+ threadIdx.y;
if(Col < w && Row < h){
int offset = Row*w + Col;
}
[CUDA 3D index]: int i=blockIdx*x*blockDim.x+threadIdx.x;
int
j=blockIdx*y*blockDim.y+threadIdx.y;
int
k=blockIdx.z*blockDim.z+threadIdx.z;
[CUDA row-major offset]: // row-major matrix M rows x
N cols; k = i*N + j; M[i][j] ->
M[i*N + j]
// column-major M rows x N cols; k = j*Mrows + i;
[Pointer arithmetic answers]: A[i][k] in row-major
flattened A: *(A + i*Ndim +
k); B[k][j] in flattened B:
*(B + k*Pdim + j)
C[i][j] write: *(C + i*Ndim + j) = tmp
[CUDA simple matrix multiply]: __global__ void
MatMulKernel(float*
M,float* N,float* P,int
Width){
int Row = blockIdx*y*blockDim.y + threadIdx.y; int Col =
blockIdx*x*blockDim.x + threadIdx.x;
if(Row < Width && Col < Width){
float Pvalue = 0; for(int k=0; k<Width; ++k)
Pvalue += M[Row*Width+k] * N[k*Width+Col];
P[Row*Width+Col] = Pvalue;
}
}
[CUDA grayscale kernel]: #define CHANNELS 3
__global__ void colorConvert(unsigned char *grayImage,
unsigned char *rgbImage,
int width, int height){
int x = threadIdx.x + blockIdx.x*blockDim.x; int y =
threadIdx.y + blockIdx.y*blockDim.y;
if(x < width && y < height){
int grayOffset = y*width + x; int rgbOffset =
grayOffset*CHANNELS;
unsigned char r = rgbImage[rgbOffset]; unsigned
char g = rgbImage[rgbOffset+1];
unsigned char b = rgbImage[rgbOffset+2];
grayImage[grayOffset] = 0.216f*r + 0.7152f*g
+ 0.0722f*b; }
}
[CUDA grayscale formula]: gray[y*w+x] = 0.216f*r +
0.7152f*g + 0.0722f*b;
rgbOffset=(y*w+x)*3.
```

[CUDA blur kernel]: `__global__ void blurKernel(unsigned char *in,unsigned char *out,int w,int h){
int Col=blockIdx.x*blockDim.x+threadIdx.x; int Row=blockIdx.y*blockDim.y+threadIdx.y;
if(Col<w && Row<h){
int pixVal=0, pixels=0; for(int br=-BLUR_SIZE;br<BLUR_SIZE+1;br++){
for(int bc=-BLUR_SIZE;bc<BLUR_SIZE+1;bc++){
int curRow=Row+br; curCol=Col+bc;
if(curRow>1 && curRow<h && curCol>-1 && curCol<w){
pixVal += in[curRow*w + curCol]; pixels++;
} }
out[Row*w+Col] = (unsigned char)(pixVal/pixels);
}
}`

[CUDA blur boundary]: For each neighbor curRow,curCol:
if(curRow>=0 && curRow<h && curCol>=0 && curCol<w)
accumulate and count pixels.

[CUDA 1D convolution kernel]: `__global__ void convolution_1D_basic_kernel(float *N,float *M, float *P,int Mask_Width,int Width){
int i = blockIdx.x*blockDim.x + threadIdx.x; float Pvalue = 0;
int N_start_point = i - (Mask_Width/2);
if(i < Width){
for(int j=0; j<Mask_Width; j++){ if(N_start_point+j >= 0 && N_start_point+j < Width)
Pvalue += N[N_start_point+j] * M[j]; P[i] = Pvalue; }
}`

[CUDA conv boundary]: `N_start=i-Mask_Width/2;
if(0<N_start+j && N_start+j<Width) Pvalue += N[N_start+j]*M[j];`

[CUDA 2D convolution kernel]: `__global__ void convolution_2D_basic_kernel(unsigned char *in, unsigned char *mask,unsigned char *out,int maskwidth,int w,int h){
int Col = blockIdx.x*blockDim.x + threadIdx.x; int Row = blockIdx.y*blockDim.y + threadIdx.y;
if(Col<w && Row<h){
int pixVal=0; int N_start_col = Col - (maskwidth/2);
int N_start_row = Row - (maskwidth/2);
for(int j=0;j<maskwidth;j++){ for(int k=0;k<maskwidth;k++){
int curRow=N_start_row+j; curCol=N_start_col+k;
if(curRow>1 && curRow<h && curCol>-1 && curCol<w)
pixVal += in[curRow*w-curCol] * mask[j*maskwidth+k];
}
} out[Row*w+Col] = (unsigned char)pixVal; }
}`

[Boundary check patterns]: 1D: if(i < Width) { ... }; 2D: if(Row < h && Col < w) { ... }
out[Row*w+Col] = ...;

Tile load missing element: `shared[ty][tx] = 0.0;`

[CUDA tiled matmul shared memory]: `__global__ void MatrixMulKernel(float *M,float *N,float *P,int Width){
__shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
__shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];
int bx=blockIdx.x, by=blockIdx.y; int tx=threadIdx.x, ty=threadIdx.y;
int Row=by*blockDim.y+ty; int Col=bx*blockDim.x+tx;
float Pvalue=0;
for(int p=0; p<(Width-1)/TILE_WIDTH+1; ++p){
if(Row<Width && p*TILE_WIDTH+tx<Width),
ds_M[ty][tx]=M[Row*Width+p*TILE_WIDTH+tx];
else ds_M[ty][tx]=0.0; if(p*TILE_WIDTH+ty<Width && Col<Width)
ds_N[ty][tx]=N[(p*TILE_WIDTH+ty)*Width+Col]; else ds_N[ty][tx]=0.0; __syncthreads();
if(Row<Width && Col<Width); for(int i=0;i<TILE_WIDTH;i++){
Pvalue += ds_M[ty][i]*ds_N[i][tx]; __syncthreads();
}
if(Row<Width && Col<Width) P[Row*Width+Col]=Pvalue;
}`

[CUDA tiled load]: M tile index: `M[Row*Width + p*TILE_WIDTH + tx];` N tile index: `N[(p*TILE_WIDTH + ty)*Width + Col];`

[CUDA tiled boundary]: If `Row=Width` or `p*TILE_WIDTH+tx>Width` set `ds_M[ty][tx]=0.0`; same for `ds_N`;
write P only if `Row/Col` valid.

[CUDA tiled matmul]: 1. declare `__shared__ ds_M` and `ds_N`. 2. Row = `blockIdx.y*blockDim.y + threadIdx`

3. Col = `blockIdx.x*blockDim.x + threadIdx`; 4. p-loop over tiles

5. load M tile and N tile with boundary checks; 6. `__syncthreads()`; multiply; `__syncthreads()`;

7. write P only if `Row<Width && Col<Width`

[CUDA shared rule]: Every thread in block must reach `__syncthreads()`; do not put `__syncthreads()` inside divergent branch unless all threads take it.

[CUDA coalescing hint]: Good: consecutive threads access consecutive memory addresses.
Row-major image: neighboring `threadIdx.x` reads in `[row*w+col]`.

[CUDA reduction complete kernel]: `#define BLOCK_SIZE 512
__global__ void total(float *input,float *output,int len){
__shared__ float partialSum[2*BLOCK_SIZE]; unsigned int t = threadIdx.idx;
unsigned int start = 2*blockIdx.x*BLOCK_SIZE; partialSum[t] = (start<len) ? input[start+t] : 0;
partialSum[BLOCK_SIZE+t] = (start<BLOCK_SIZE+len) ? input[start+BLOCK_SIZE+t] : 0;
for(unsigned int stride=BLOCK_SIZE; stride>=1; stride>=1){
__syncthreads(); if(t < stride) partialSum[t] += partialSum[t+stride]; }
if(t == 0) output[blockIdx.x] = partialSum[0];
}`

[CUDA reduction host code]: `numOutputElements = numInputElements / (BLOCK_SIZE << 1);
dim3 dimGrid(numOutputElements,1,1);
dim3 dimBlock(BLOCK_SIZE,1,1);
total<<<dimGrid,dimBlock>>>(deviceInput,deviceOutput, numInputElements);
cudaDeviceSynchronize();
cudaMemcpy(hostOutput,deviceOutput,sizeof(float)* numOutputElements, cudaMemcpyDeviceToHost);`

[CUDA reduction stride]: partialSum has 2*BLOCK_SIZE. Load two elements per thread, then for(stride=BLOCK_SIZE; stride>=1; stride>=1).

[CUDA reduction race]: Need `__syncthreads()` before each stride because thread t reads `partialSum[t+stride]` produced in previous stride.

[Reduction blank answers]: Need `__syncthreads()` before reading `partialSum[t+stride]`;
Use `stride >= 1` to halve stride.

Only three t=0 writes `output[blockIdx.x]`; `partialSum` length = 2*BLOCK_SIZE.

[CUDA streams overlap pattern]: `cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);
cudaMemcpyAsync(d1,h1.bytes,cudaMemcpyHostToDevice, stream1);
kernel<<<blocks,threads,sharedMem,stream1>>>(d1);
cudaMemcpyAsync(d2,h2.bytes,cudaMemcpyHostToDevice ,stream2);
kernel<<<blocks,threads,sharedMem,stream2>>>(d2);
cudaDeviceSynchronize();`

[CUDA stream in cuSOLVER]: `cusolverDnHandle_t cusolverH=NULL;
cudaStream_t stream=NULL;
status = cusolverDnCreate(&cusolverH);
cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
cusolverDnSetStream(cusolverH, stream);`

[CUDA async copies]: `cudaMemcpyAsync` requires stream argument; overlap works best with pinned host memory and independent streams.

[CUDA device struct with pointer]: `struct vector { double *a; int n; };
cudaMalloc(&d_vectors, N*sizeof(struct vector));
cudaMemcpy(d_vectors,h_vectors,N*sizeof(struct vector),H2D);
for(int i=0;i<N;i++){
double *d_a;
cudaMalloc(&d_a,h_vectors[i].n*sizeof(double));
cudaMemcpy(d_a,h_vectors[i].h_vectors[i].n*sizeof(double),H2D);
cudaMemcpy(&(d_vectors[i].a), &d_a, sizeof(double*), H2D);
}`

[CUDA pointer struct rule]: Copying struct with host pointer only copies bad host address; allocate device pointer and patch device struct pointer field.

[CUDA dims limits]: Max threads/block = 1024; Valid: `dim3 block(32,32,1) -> 1024`; Invalid: `dim3 block(32,32,2) -> 2048`.

Grid/block launch: `dim3 DimGrid(kg,jg,ig);
dim3 DimBlock(kb,jb,ib);
kernel<<<DimGrid,DimBlock>>>(arg1,arg2);
[CUDA max threads]: blockx * blocky * blockz <= 1024. Common 2D blocks: 16x16+256, 32x8+256, 32x32+1024.`

[CUDA mem types]: register: per block, fast, lifetime block; global: device DRAM, grid/app lifetime; constant: read-only broadcast.

[CUDA managed sync]: `cudaMallocManaged(&a,bytes);
kernel<<<...>>>(a);
cudaDeviceSynchronize(); CPU reads a; cudaFree(a);`

[Common CUDA offload checklist]: 1. `cudaMalloc` all device arrays; 2. `cudaMemcpy` input H2D; 3. launch `kernel<<<grid,blocks>>>(...)`

4. `cudaDeviceSynchronize()` when needed; 5. `cudaMemcpy` output D2H; 6. `cudaFree` all device arrays

7. check errors after kernel

[CUDA profile flow]: 1. `cuda-memcheck ./prog 2 nsys profile ./prog 3 ncu --set full ./prog`
4. compare kernel time + memory throughput

[CUDA Thrust add]: `thrust::copy(h.begin(),h.end(),d.begin());
thrust::transform(d1.begin(),d1.end(),d2.begin(),out.begin(),thrust::plus<float>());`

[HIP mirror names]: `cudaMalloc->hipMalloc;
cudaMemcpy->hipMemcpy;
cudaDeviceSynchronize->hipDeviceSynchronize;
__global__ kernel syntax same.`

[CSR arrays example]: A has m=4,n=5,nnz=8; `csr_val = {1,2,3,4,5,6,7,8}; csr_col_ind = {0,1,3,1,2,0,3,4}`
`csr_row_ptr = {0,3,5,5,8};` row i uses `k = row_ptr[i] - row_ptr[i-1]`
row 2 has zero nnz because `row_ptr[2]=row_ptr[3]`

[CSR dense-to-CSR build]: `int ptr=0;
for(int i=0;i<size;i++){
hRowPtr.push_back(ptr);
for(int j=0;j<size;j++){
if(hA[i*size+j] != 0){
hVal.push_back(hA[i*size+j]);
hColIndex.push_back(j); ptr++; }
}`

[CSR row count]: `row_nnz = csr_row_ptr[i+1]-csr_row_ptr[i];` empty row if equal; last `csr_row_ptr[m]=nnz`

[Sparse blank answers]: `nnz = hRowPtr.back();` row i count = `csr_row_ptr[i+1] - csr_row_ptr[i]`; col = `csr_col_ind[k]`
val = `csr_val[k]`; CSR better than dense when zeros dominate.

[CSR decode pattern]: `for(i=0;i<m;i++){ for(k=hRow_ptr[i]; k<row_ptr[i+1]; k++) j=col_ind[k]; dense[i* n]=val[k]; }`

[rocsPARSE alloc/copy CSR]: `float *dB,*dC,dVal; int *dRowPtr,*dColInd;
hipMalloc((void**) &dB,sizeof(float)*size*size);
hipMalloc((void**) &dC,sizeof(float)*size*size);
hipMalloc((void**) &dVal,sizeof(float)*nnz);
hipMalloc((void**) &dRowPtr,sizeof(float)*(size+1));
hipMalloc((void**) &dColInd,sizeof(float)*nnz);
hipMemcpy(dVal,hVal,data(),sizeof(float)*nnz,H2D);
hipMemcpy(dRowPtr,hRowPtr,data(),sizeof(float)*(size+1), H2D);
hipMemcpy(dColInd,hColIndex,data(),sizeof(float)*nnz,H2D);`

[rocsPARSE csrmv call]: `rocsparse_handle handle; rocsparse_create_handle(& handle);
rocsparse_mat_descr descr;
rocsparse_create_mat_descr(&descr);
float alpha=1.0, beta=0.0; status = rocsparse_scsrmv(handle, rocsparse_operation_none, rocsparse_operation_transpose, size, size, nnz, &alpha, descr, dVal, dRowPtr, dColInd, dB, size, &beta, dC, size);
if(status != rocsparse_status_success) exit(2);`

[CSR SpMM meaning]: `C = alpha*op(A)*op(B) + beta*C`; A sparse CSR, B dense, C dense in rocsPARSE csrmv.

[cuSPARSE CSR add setup]: `// C = alpha*A + beta*B, all CSR, nnzA/nnzB/nnzC differ
cusparseHandle_t handle; cusparseCreate(&handle);
cusparseSetPointerMode(handle,CUSPARSE_POINTER_MODE_HOST); const float alpha=1.0, beta=1.0;
cusparseMatDescr_t descr=0, descrB=0, descrC=0;
cusparseCreateMatDescr(&descr);
cusparseCreateMatDescr(&descrB);
cusparseCreateMatDescr(&descrC);`

[cuSPARSE CSR add nnz]: `cusparseScsgeam2_bufferSizeEx(handle,size,size,alpha, nnzA,dVal,dRowPtr,dColInd,&beta,descrB,nnzB,dBVal,dBRowPtr, dBColInd,descrC,dCVal,csrRowPtrC,dCColInd,&pBufferSizeInBytes);
cusparseXcsgeam2Nnz(handle,size,size,descrnnzA, dRowPtr,dColInd, descrB,nnzB,dBRowPtr,dBColInd,descrC,csrRowPtrC, nnzTotalDevHostPtrpBuffer);`

[cuSPARSE CSR add compute/copy]: `if(nnzTotalDevHostPtr) nnzC =
r;
else { cudaMemcpy(&nnzC, csrRowPtrC+size, sizeof(int), D2H);
nnzC-=baseC; }
cudaMalloc((void**) &dCColInd,sizeof(int)*nnzC);
cudaMalloc((void**) &dCVal,sizeof(float)*nnzC);
cusparseScsgeam2(handle,size,size,alpha,descrnnzA, dVal,dRowPtr,dColInd, &beta,descrB,nnzB,dBVal,dBRowPtr,dBColInd,descrC,dCVal, csrRowPtrC,dCColInd,pBuffer);
cudaMemcpy(hCVal,data(),dCVal,sizeof(float)*nnzC,D2H);`

[cuSOLVER LU declarations]: `cusolverDnHandle_t cusolverH=NULL;
cudaStream_t stream=NULL;
int int m=3, ldm=m, ldb=m; double A[lda*m] = {1,4,7, 2,5,8, 3,6,10}; // column-major
double B[m] = {1,2,3}; double X[m], LU[lda*m]; int Ipv[m], info=0;
double *dA=NULL,*dB=NULL,*d_work=NULL; int *d_Ipv=NULL,*d_info=NULL; int lwork=0;
[cuSOLVER LU alloc/copy]:
cudaMalloc((void**) &dA, sizeof(double)*lda*m);
cudaMalloc((void**) &dB,sizeof(double)*m);
cudaMalloc((void**) &d_Ipv,sizeof(int)*m);
cudaMalloc((void**) &d_info,sizeof(int));
cudaMemcpy(dA,A,sizeof(double)*lda*m,H2D);
cudaMemcpy(dB,B,sizeof(double)*m,H2D);`

[cuSOLVER LU call pattern]: `cusolverDnGetrf_bufferSize(cusolverH,m,m,dA,lda &lwork);
cudaMalloc((void**) &d_work,sizeof(double)*lwork);
cusolverDnGetrf(cusolverH,m,m,dA,lda,d_work,d_Ipv, d_info);
cusolverDnDgtrs(cusolverH,CUBLAS_OP_N,m,dA,lda, d_Ipv,dB,ldb,d_info);
cudaMemcpy(X,dB,sizeof(double)*m,D2H);`

[cuSOLVER column-major]: `A[lda*m]={1,4,7, 2,5,8, 3,6,10}` stores columns: `column0={1,4,7}, column1={2,5,8}`

[cuSOLVER getrf/getrs]: `getrf` computes LU (+ pivots); `getrs` solves using LU factors. Always check `d_info` copied back to host.

[cuSOLVER eig alloc/buffer]: `double V[lda*m], W[m];
double *d_A=NULL,*d_W=NULL,*d_work=NULL;`

`int *devInfo=NULL, lwork=0, info_gpu=0;
cudaMalloc((void**) &dA,sizeof(double)*lda*m);
cudaMalloc((void**) &dW,sizeof(double)*m);
cudaMalloc((void**) &devInfo,sizeof(int));
cudaMemcpy(dA,A,sizeof(double)*lda*m,H2D);
cusolverEigMode_t jobz=CUSOLVER_EIG_MODE_VECTOR;
cublasFillMode_t uplo=CUBLAS_FILL_MODE_LOWER;
cusolverDnDsyevd_bufferSize(cusolverH,jobz,uplo,m,dA, lda,d_W,&lwork);
cudaMalloc((void**) &d_work,sizeof(double)*lwork);`

[cuSOLVER eig compute]: `cusolverDnDsyevd(cusolverH, jobz,uplo,m,dA,lda,d_W, d_work,work,devInfo);
cudaDeviceSynchronize();
cudaMemcpy(Wd,W,sizeof(double)*m,D2H);
cudaMemcpy(Vd,A,sizeof(double)*lda*m,D2H);
cusolverDnDsyevd_bufferSize(cusolverH,jobz,uplo,m,dA, lda,d_W,&lwork);`

`for(int row=0; row<m; row++){
printf("lambda %d = %f\n", row+1, W[row]);
cudaFree(dA); cudaFree(d_W); cudaFree(devInfo);
cudaFree(d_work);`

[cuSOLVER eig]: `Dsyevd` overwrites `dA` with eigenvectors, writes eigenvalues to `d_W`. call `bufferSize` before allocating `d_work`.

[roCFFT setup/plan]: `#include <hip_runtime.h>
#include <roCfft/roCfft.h>
roCfft_setup(0);
size_t N=16, Nbytes=N*sizeof(float2); float2 *x;
hipMalloc(&x,Nbytes); std::vector<float2> cx(N);
for(size_t i=0;i<N;i++){ cx[i].x=1, cx[i].y=-1; }
hipMemcpy(x,cx,data(),Nbytes,H2D);
roCfft_plan plan=roCfft; size_t length=N;
roCfft_plan_create(&plan,roCfft_placement_inplace, roCfft_transform_type_complex_forward, roCfft_precision_single,1,&length,1,nullptr);`

`[roCFFT execute/cleanup]: size_t work_buf_size=0;
roCfft_plan_get_work_buffer_size(plan,&work_buf_size);
void *work_buf=nullptr; roCfft_execution_info info=nullptr;
if(work_buf_size){
roCfft_execution_info_create(&info);
hipMalloc(&work_buf,work_buf_size);
roCfft_execution_info_set_work_buffer(info,work_buf, work_buf_size);
}`

`roCfft_execute(plan,(void**) &x,nullptr,info);
hipDeviceSynchronize();
if(work_buf_size){ hipFree(work_buf);
roCfft_execution_info_destroy(info); }
roCfft_plan_destroy(plan); roCfft_cleanup();`

[roCFFT lifecycle]: `roCfft_setup(0); roCfft_plan_create(...);
roCfft_execute(...);
roCfft_plan_destroy(plan);
roCfft_cleanup();`

[roCRAND generator]: `#include <rocrand/rocrand.h>
size_t n=1048576;
rocrand_generator gen;
float *d_rand,"h_rand";
h_rand=(float*)malloc(sizeof(float)*n);
hipMalloc((void**) &dRand,n*sizeof(float));
rocrand_create_generator(&gen,ROCRAND_RNG_PSEUDO_0_DEFAULT);
rocrand_set_seed(gen,time(NULL));
rocrand_generate_uniform(gen,dRand,n,rocrand,rocrand_destroy_generator(gen),hipFree(dRand);`

[float precision code check]: `double u = pow(2,-53);
printf("1.0 + 2^-53 == 1.0 ? %s\n", (1.0+u==1.0)?"true":"false");
printf("1.0 + 1e-16 == %17f\n", 1.0+1e-16);
printf("1.0 + 2e-16 == %17f\n", 1.0+2e-16);`

Use tolerance in comparisons: `fabs(a-b) < TOL`

[FP32 decode formula]: `// value = (-1)^sign * (1 + fraction) * 2^(exponent-bias);
FP32 bias = 127; FP16 bias = 15;
Example sign=1, exponent=129, fraction=25; value = -1 * 125 * 2^(129-127) = -5.0`

[FP compare]: Never compare floats with `==` for numerical kernels; use `fabs(gpu-cpu) < tolerance` or relative error.

[FP32 decode]: `value=(-1)^sign*(1+fraction)*2^(exponent-127)`; exponent bits determine range, mantissa bits determine precision.

[CUDA linearize 3D]: `idx = (k*ny + j)*nx + i; // i fastest, then j, then k; guard <nx && jny && k<nz`

[OpenMP exam fill]: `#pragma omp parallel for default(none) shared(N,A,B,C) private(i) reduction(+,sum)`

[CUDA exam fill]: `int i=blockIdx.x*blockDim.x+threadIdx.x; if(i<N) out[i]=in[i];`